

Schema Evolution

.....

Big Data Systems
School of Data Science
University of Virginia

What is a Schema?

A schema is a contract between producers and consumers of data.
It is a formal description of:

- > Structure (fields, types, relationships)
- > Constraints (required vs optional)
- > Semantics (what the data means)

Examples of Schema

Database table definitions

API request/response formats

Event payloads

Serialized objects

Why do Schemas Evolve?

Schemas evolve because systems evolve:

- > New features require new data
- > Old fields become obsolete
- > Requirements change over time
- > Different teams move at different speeds

Why this Matters

Schema evolution affects:

- > Databases
- > Distributed systems
- > Streaming pipelines
- > Analytics & ML
- > Compliance and auditing

Most Important Thing

Stable systems require changing schemas without things breaking.

In a past role, the most common cause of a breakage was the dreaded “schema change”

Backward Compatibility

New code can read old data.

New versions don't break existing users.

Examples:

- > Adding an optional field
- > Providing defaults for missing fields



Forward Compatibility

Old version can handle new data/features without breaking.

Example:

- > Ignoring unknown fields
- > Avoiding required new fields

Common Schema Changes

By Risk Level

Usually safe

- > Add optional field
- > Add new enum value (restricted set of values)
- > Relax constraints

Common Schema Changes, contd.

By Risk Level

Risky

- > Remove fields
- > Rename fields
- > Change meaning without changing name

Common Schema Changes, contd.

By Risk Level

Dangerous

- > Change data type
- > Change field semantics silently
- > Reorder fields in positional formats

Schema Evolution Strategies

Additive

Prefer adding fields over modifying/removing

Mark old fields as deprecated

Pros: safe, incremental

Cons: schemas grow over time

Schema Evolution Strategies

Versioned schemas

- > Explicit version numbers
- > Versioned endpoints or files
- > Schema registries
 - *Centralized service* that stores and manages data schemas
 - Helps data producers & consumers agree on structure, compatibility

Schema Registry

How it works:

- 1 Stores schemas centrally
- 2 Assigns version numbers
- 3 Enforces compatibility rules
- 4 Validates new schema updates
- 5 Allows consumers to fetch correct schema

Schema Registry Example

Version 1:

```
json
{
  "userId": "string",
  "email": "string"
}
```

Version 2 (safe change):

```
json
{
  "userId": "string",
  "email": "string",
  "phone": "string (optional)"
}
```

If compatibility mode set to BACKWARD, registry will:

- > Allow adding optional fields
- > Reject breaking changes (like removing required fields)

Schema Evolution Strategies, contd.

Transformation layers

- > Adapters or translators
- > Migrate data at read or write time

Used when:

- > Legacy data must coexist
- > Hard breaks are unavoidable

Schema-on-write

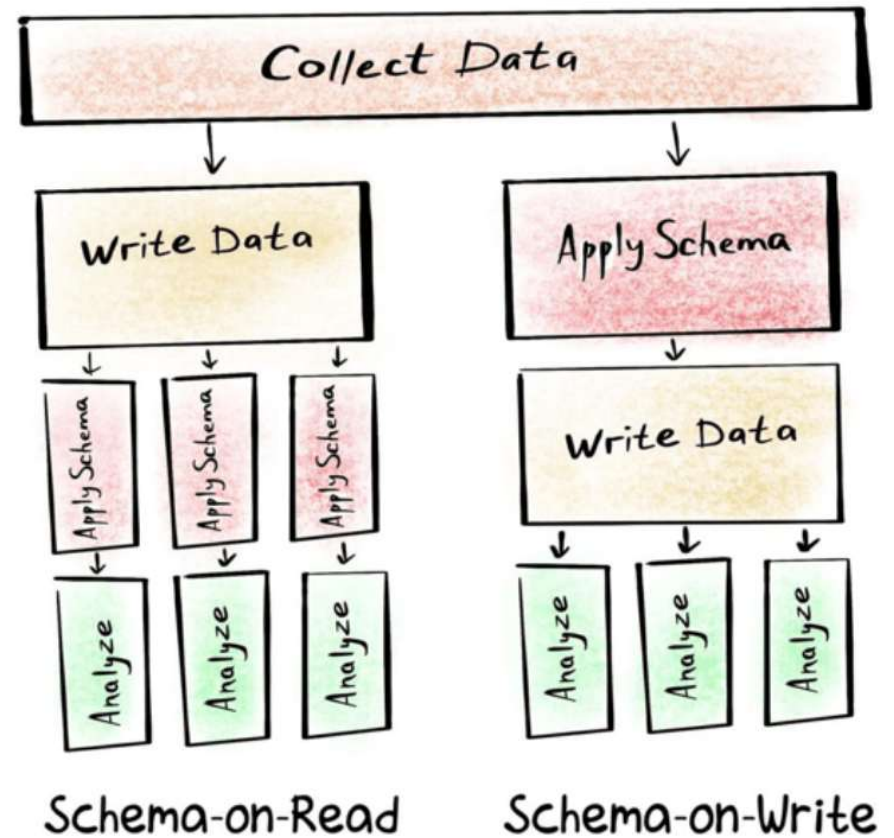
Enforce schema before data is stored

Reject invalid data

Pros: data quality

Cons:

- > harder evolution
- > need to understand data to define schema



Schema-on-read

Interpret data when consumed

Flexible, tolerant of change

Pros: agility

Cons: complexity shifts downstream

Key idea:

Evolution strategy depends on where schema is enforced.

Failure Modes

- > Silent corruption
- > Partial compatibility
- > Inconsistent interpretations
- > Long-tail bugs from old data (edge / rare cases that show much later)

Worst case:

Systems work — but answers are wrong.



Design Principles (Takeaways)

Treat schemas as public APIs

Prefer backward-compatible changes

Version explicitly when breaking

Document semantics, not just structure

Plan for old data to live forever